

# Revision 05-07 • Attention Math + Transformer Architecture

Interactive revision of [lesson 05](#) and [lesson 07](#): each question as asked, the answer, where I slipped, and the one-line takeaway.

## Q1 • Derive KV bytes per token, and what GQA buys SOLID

Covers: 05, 07, 19

### QUESTION

Llama 3 8B: 32 layers, 8 KV heads, head dim 128, BF16. How many bytes of KV cache per token? Llama 2 7B is the same but with 32 KV heads (no GQA): what's its KV per token, and what does that do to users per GPU?

### How I solved it (powers of two)

1. KV/token = 2 (K and V)  $\times$  32 (layers)  $\times$  8 (KV heads)  $\times$  128 (head dim)  $\times$  2 (bytes, BF16).
2. In powers of two:  $2^1 \times 2^5 \times 2^3 \times 2^7 \times 2^1 = 2^{17} = 131,072$  bytes = **128 KiB**.
3. Llama 2 7B swaps 8 heads ( $2^3$ ) for 32 ( $2^5$ ):  $2^{19} =$  **512 KiB**, 4 $\times$  more.
4. Same H100 and  $\sim 53$  GB cache pool (lesson 18): 4 $\times$  the bytes per token means **4 $\times$  fewer tokens**,  $\sim 100k$  instead of  $\sim 400k$ , so  $\sim 25$  users at 4k context instead of  $\sim 100$ .

### RE-DRILL

Powers of two make these products instant:  $32 = 2^5$ ,  $8 = 2^3$ ,  $128 = 2^7$ , and  $2^{10} = 1$  KiB.

## Q2 • Parameters in one transformer block PARTIAL

Covers: 07, 12

### QUESTION

Llama 3 8B:  $h = 4,096$ , MLP size  $i = 14,336$ , KV width  $8 \times 128 = 1,024$ . Q and O are  $h \times h$ ; K and V are  $h \times 1,024$ ; the MLP has three  $h \times i$  matrices. How many parameters per block, and which part dominates?

**Answer** (asked for the worked answer;  $h = 2^{12}$ , KV width =  $2^{10}$ ,  $i = 3.5 \times 2^{12}$ )

| Part                        | Matrices                                                                              | Count                                                         |
|-----------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Q + O                       | $2 \times h \times h = 2 \times 2^{12} \times 2^{12} = 2^{25}$                        | $\approx 33.6\text{M}$                                        |
| K + V                       | $2 \times h \times 1,024 = 2 \times 2^{12} \times 2^{10} = 2^{23}$                    | $\approx 8.4\text{M}$                                         |
| <b>Attention</b>            |                                                                                       | <b><math>\approx 42\text{M}</math></b>                        |
| <b>MLP</b> (gate, up, down) | $3 \times h \times i = 3 \times 2^{12} \times 3.5 \times 2^{12} = 10.5 \times 2^{24}$ | <b><math>\approx 176\text{M}</math></b>                       |
| <b>Block</b>                |                                                                                       | <b><math>\approx 218\text{M}</math></b> (MLP $\approx 81\%$ ) |

- Check:  $32 \times 218\text{M} \approx 6.98\text{B}$ , plus embedding + LM head  $2 \times 128,256 \times 4,096 \approx 1.05\text{B} \rightarrow \mathbf{8.03\text{B}}$ .
- **Why it matters:** decode time = bytes  $\div$  bandwidth and the MLP holds  $\sim 81\%$  of each block's bytes, so it's the first target for quantization and for MoE. Speeding up attention alone barely helps (Amdahl).

**RE-DRILL**

Re-drill the shape of each matrix: Q, O are  $h \times h$ ; K, V are  $h \times (\text{KV heads} \times \text{head dim})$ ; the MLP is three  $h \times i$ .

### Q3 · Is MoE cheap to host? (Mixtral 8x7B) PARTIAL

Covers: 07, 18

**QUESTION**

"Mixtral 8x7B only uses  $\sim 13\text{B}$  parameters per token, so host it like a 13B model on one 80 GB H100 in BF16." Mixtral stores 46.7B parameters, 12.9B active per token. Right or wrong, for memory and for compute?

**Answer**

- **Memory follows total parameters:**  $46.7\text{B} \times 2 \text{ bytes} \approx \mathbf{93 \text{ GB}}$  in BF16  $> 80 \text{ GB}$ , so **it doesn't fit**. Options: FP8 ( $\sim 47 \text{ GB}$ , leaving  $\sim 72 - 47 - 3 \approx 22 \text{ GB}$  for KV cache) or 2 GPUs.
- **Compute follows active parameters:**  $2 \times 12.9\text{B} \approx \mathbf{26 \text{ GFLOPs per token}}$ , like a 13B model. That part is true.
- **Nuance:** at batch 1, decode reads only the active experts' weights, so it's fast like a 13B. In a big batch, tokens route to different experts, so nearly all 93 GB gets read each step and the advantage shrinks.

**RE-DRILL**

Always end a "can we?" question with the number and a yes/no.

### Q4 · Why $\div \sqrt{d}$ , and why subtract the max in softmax? PARTIAL

Covers: 05, 13

**QUESTION**

Attention computes  $q \cdot k \div \sqrt{d}$  before softmax. What goes wrong without  $\div \sqrt{d}$  ( $d = 128$ )? Softmax kernels subtract the row's maximum before  $e^x$ : why, and does it change the result?

**Answer**

- **$\div \sqrt{d}$ :** a dot product of  $d$  random unit-variance numbers has variance  $d$ , so scores spread by  $\sqrt{128} \approx 11$ . Dividing by  $\sqrt{d}$  brings the spread back to  $\sim 1$ . Without it, softmax saturates into **nearly one-hot**: one token takes all the weight, attention stops blending, and gradients vanish in training.
- **Subtract the max:** prevents **overflow**: FP16 tops out at 65,504, and  $e^{12} \approx 162,755$  already overflows.
- **Same result:**  $e^{(x - m)} = e^x \cdot e^{-m}$ , and the  $e^{-m}$  cancels between numerator and denominator. Only differences between scores matter (lesson 13).

**RE-DRILL**

For every "why do we do X?" give both halves: what breaks without it, and what it costs (here: nothing).

**Exercise · Online softmax and one-pass attention** **PARTIAL**

Covers: 05

**QUESTION**

FlashAttention sees scores in blocks. Compute exact softmax weights block by block (Part A), then the attention output  $\sum \text{softmax}_i \times v_i$  in one pass without storing the weights (Part B).

**The idea:** keep a running max  $m$  and a running sum  $l$  of  $e^{(\text{score} - m)}$ . When a block raises the max, rescale everything summed so far by  $e^{(m_{\text{old}} - m_{\text{new}})}$ , because  $e^{(x - m_{\text{old}})} \times e^{(m_{\text{old}} - m_{\text{new}})} = e^{(x - m_{\text{new}})}$ . For attention, keep  $\text{acc} = \sum e^{(\text{score} - m)} \times \text{value}$ , rescaled the same way; the output is  $\text{acc} / l$ .

**Worked example:** blocks  $[2, 1]$  then  $[3, 0]$ . After block 1:  $m = 2, l = e^0 + e^{-1} \approx 1.368$ . Block 2:  $m = 3, l = 1.368 \times e^{-1} + e^0 + e^{-3} \approx 0.503 + 1.050 = 1.553$ , the same as the full row.

```
def online_attention(scores, values, block_size):
    m, l, acc = float("-inf"), 0.0, 0.0 # running max, denominator, numerator × values
    for start in range(0, len(scores), block_size):
        block_s = scores[start:start + block_size]
        block_v = values[start:start + block_size]
        m_new = max(m, max(block_s)) # never decreases
        scale = math.exp(m - m_new) # re-express old sums against the new max
        l = l * scale + sum(math.exp(s - m_new) for s in block_s)
        acc = acc * scale + sum(math.exp(s - m_new) * v for s, v in zip(block_s, block_v))
        m = m_new
    return acc / l

online_attention([2, 1, 3, 0], [10, 20, 30, 40], 2) # 24.71, same as the naive version
```

**RE-DRILL**

Every running quantity measured "relative to  $m$ " must be rescaled together when  $m$  changes.